

第 9 章 检索增强生成 —— 课程实践作业实验报告

《大规模语言模型：从理论到实践》· 第 9 章 检索增强生成

1. 任务与分工

本作业的目标不是“跑通一个 RAG”，而是拆解索引 → 检索前优化 → 检索 → 检索后优化 → 生成 → 编排的每一个模块，用实验数据说明每个模块“有没有用、在什么情况下有用”。本次完成范围：T1-T5 为必做实验，T7 系统评估（选做其一），T8-1 幻觉治理为加分项。

成员	负责模块	对应文件
刘嘉豪-12026219156	索引构建与分块、检索策略、评测集构造、报告撰写、生成、重排序与压缩、评估脚本、PPT	src/index.py、src/retriever.py、data/eval/qa_set.jsonl、src/generate.py、src/rerank.py、src/evaluate.py、scripts/run_t*.py

2. 数据说明

2.1 自建语料 (D2)

来源与规模：语料为 LangChain 官方文档 (docs.langchain.com) 中与第 9 章高度相关的 v1 新文档，自建抓取并做中文整理，抓取日期 2026-09-26；共 26 篇 Markdown，约 3.08 万字符 (49.9 KB)。

主题分布：按目录分为 5 类——01_architecture（框架架构）、02_rag_retrieval（RAG 检索）、03_middleware（中间件）、04_integrations（集成）、05_extensions（扩展与评估）。

选择理由：与课件“9.6 LangChain 实践”直接对应；其中 v1 新特性、迁移指南等内容时效性强、大概率不在大模型训练数据中，便于观察 RAG 的真实增益。

切分方式：默认按 Markdown 的 ## 标题切分，共 139 个 chunk，chunk_id 形如 doc_id#c001。

2.2 自建评测集 (D3)

评测集为 data/eval/qa_set.jsonl，共 25 条，依据课件 9.1.2 的 RAG 任务分级构造，四个等级各不少于 5 条，且每条都标注 gold_chunk_ids 用于计算 Recall@k。

等级	说明	条数
L1	显性事实查询（答案明文在单一片段）	6
L2	隐性事实查询（跨片段或简单常识推理）	6
L3	可解释推理查询（需依据外部规则）	6
L4	隐性推理查询（需领域经验、开放式）	7

每条评测样本包含字段：id、level、question、gold_answer（人工核对的标准答案）、gold_chunk_ids（支撑答案的片段编号）、evidence_required（是否需要证据）。L4 类问题允许要点式评分标准。

3. 系统架构

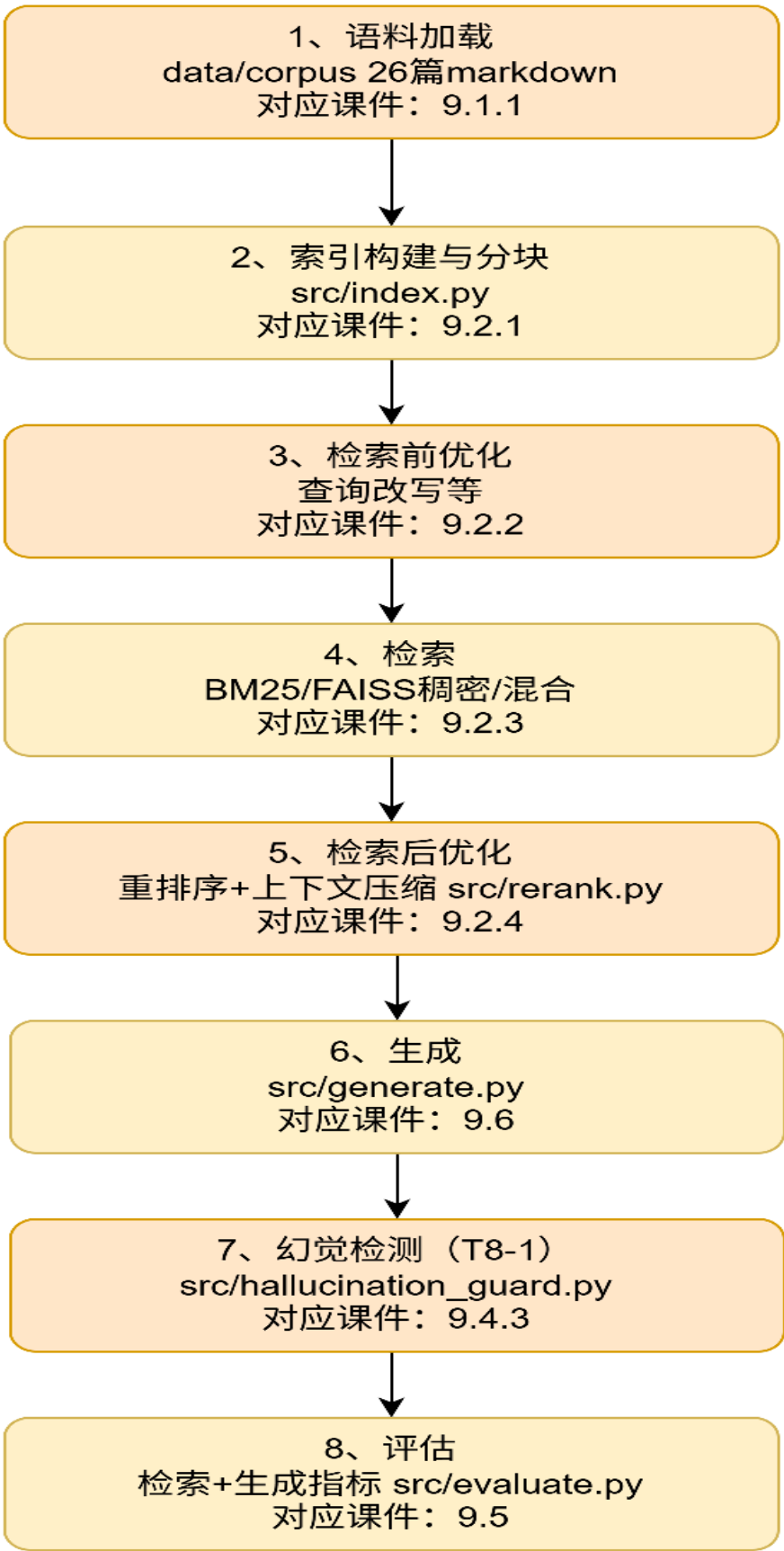


图 1 系统整体流程图（模块与课件章节对应）

模块	实现	技术选型 / 关键参数
LLM 生成	通义千问 qwen-plus（DashScope OpenAI 兼容接口）	temperature=0.0
Embedding	text-embedding-v3（DashScope）	单次批量 ≤ 10 条
向量库	FAISS（CPU）	余弦/内积相似度检索
稀疏检索	BM25（rank_bm25）	中英文按字符/词切分
重排	Cross-Encoder bge-reranker-base / 规则 / LLM Judge	top-5 → top-3
融合与参数	RRF（k=60）	top-k=5、seed=42、默认 chunk 512/64

整体流程与课件章节的对应关系已标注在图 1 中：索引构建对应 9.2.1，检索前优化对应 9.6（p39–40）与 9.2.2，检索对应 9.2.3，检索后优化对应 9.2.4，生成对应 9.6，幻觉检测对应 9.4.3，评估对应 9.5（p33–36）。

4. 实验与结果

4.1 T1 基线 RAG 与裸 LLM 对比

基线链路：按 ## 标题切分 → FAISS 稠密检索 top-5 → qwen-plus 生成；对照组不接检索、直接用同一 LLM 回答同样问题。对评测集 25 条全部产出答案，保存到 results/t1_baseline.jsonl 与 results/t1_bare_llm.jsonl。

成功案例（2 例）：

q001（L1，create_agent 导入路径）：RAG 输出 “from langchain.agents import create_agent”（正确）；裸 LLM 却写成 initialize_agent / Agent.from_llm_and_tools 等 v0.x 用法，并断言 create_agent 不存在，明显混淆 v1 与 v0.x，产生事实性幻觉。

q004（L1，PIIMiddleware 处理策略）：RAG 输出 “redact、mask、hash、block”（正确）；裸 LLM 却声称“PIIMiddleware 不是标准中间件”，同样因未见过 v1 文档而否认其存在。

对照：q005（L1，2-Step 与 Agentic RAG）属通用概念，裸 LLM 也能答对；裸 LLM 的失效集中在 create_agent、PIIMiddleware、SummarizationMiddleware 等 v1 新 API，恰好是“模型没背过”的时效性内容。

失败案例（2 例）：

q024（L4，LangSmith 如何支撑失败归因）：检索未召回 06_tracing 的具体追踪内容片段（gold c004/c005），转而召回评估/指标类片段，最终回答“根据提供的资料无法回答”——属检索失败。

q019（L4，v0.x→v1 迁移风险与收益）：已召回迁移指南相关片段（含 gold c001），但生成仍拒绝回答——属“召回了但生成失败”。

结论：裸 LLM 对通用知识（如 q005）能正确回答，但对 v1 新 API/新架构这类时效性内容系统性失效（幻觉或否认其存在）；基线 RAG 恰好补上这块并支持 chunk 溯源。剩余问题集中在 L3/L4 复杂问题的召回不足与生成拒答。

4.2 T2 索引模块：分块策略实验

固定检索与生成不变，只改索引阶段的切分方式，做“切分方法 × chunk_size × overlap”网格实验，以 Recall@5 为主指标。结果见下表与图 2。

切分方法	chunk_size	chunk_overlap	chunk 数	Recall@5
header	512	0	139	0.56
recursive	256	0	165	0.80
recursive	256	128	193	0.72
recursive	512	0	81	0.60

recursive	512	128	86	0.52
recursive	1024	0	42	0.36
recursive	1024	128	42	0.44
sliding	256	0	133	0.88
sliding	256	128	229	0.56
sliding	512	0	75	0.48
sliding	512	128	84	0.56
sliding	1024	0	41	0.36
sliding	1024	128	43	0.40

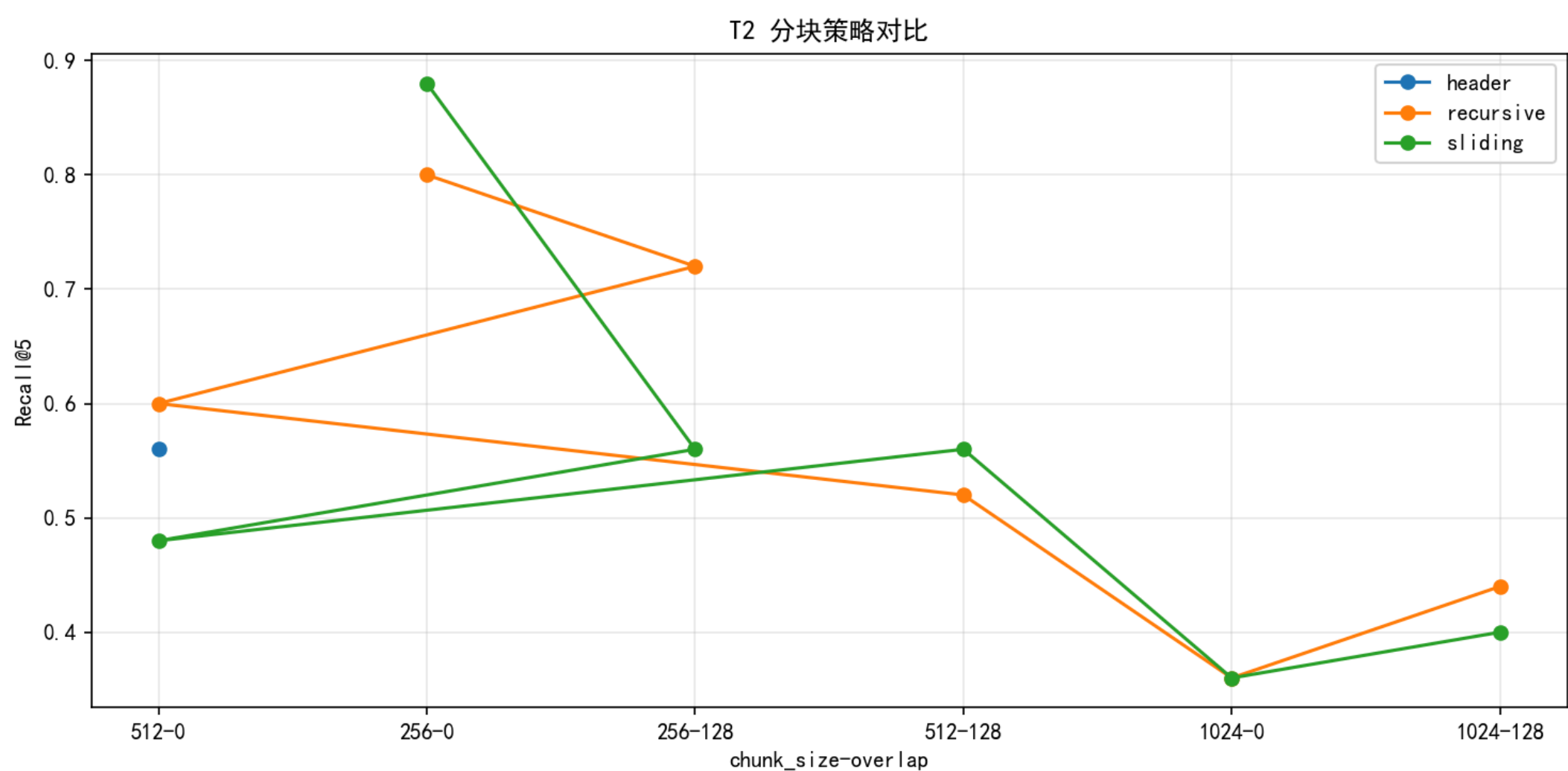


图 2 T2 分块策略 Recall@5 对比

最优配置为 sliding-256-0 (Recall@5 = 0.88)；1024 大块普遍降至 0.36–0.44；256 小块整体优于 512/1024。

overlap=128 未带来一致收益：sliding-256-128 反而降到 0.56 (chunk 数从 133 增至 229，重复内容变多)。

归因：语料是短小、主题分明的技术文档，“一段一主题”的小块 + 无重叠滑动窗口最贴合；大块会把多个 ## 主题合并进同一 chunk，噪声变大。例如 recursive-1024 下某文档多个小节被合并，gold 片段与大量无关内容共存，Recall 降至 0.36。

注：本网格为控制 LLM 打分成本，未逐条计算答案正确性（脚本支持 `-skip-answer`），故 T2 结论以 Recall@5 为依据。

4.3 T3 检索前优化：查询分解与融合

实现查询改写 (rewrite)、多查询 (multi_query)、子查询分解 (sub_query) 与 RRF 融合，在评测集上对比 Recall@5。

策略	Recall@5
baseline（原始查询）	0.56
query rewrite（改写）	0.64
multi-query（多查询）	0.64
sub-query（子查询分解）	0.72
multi-query + dense + sparse	0.60

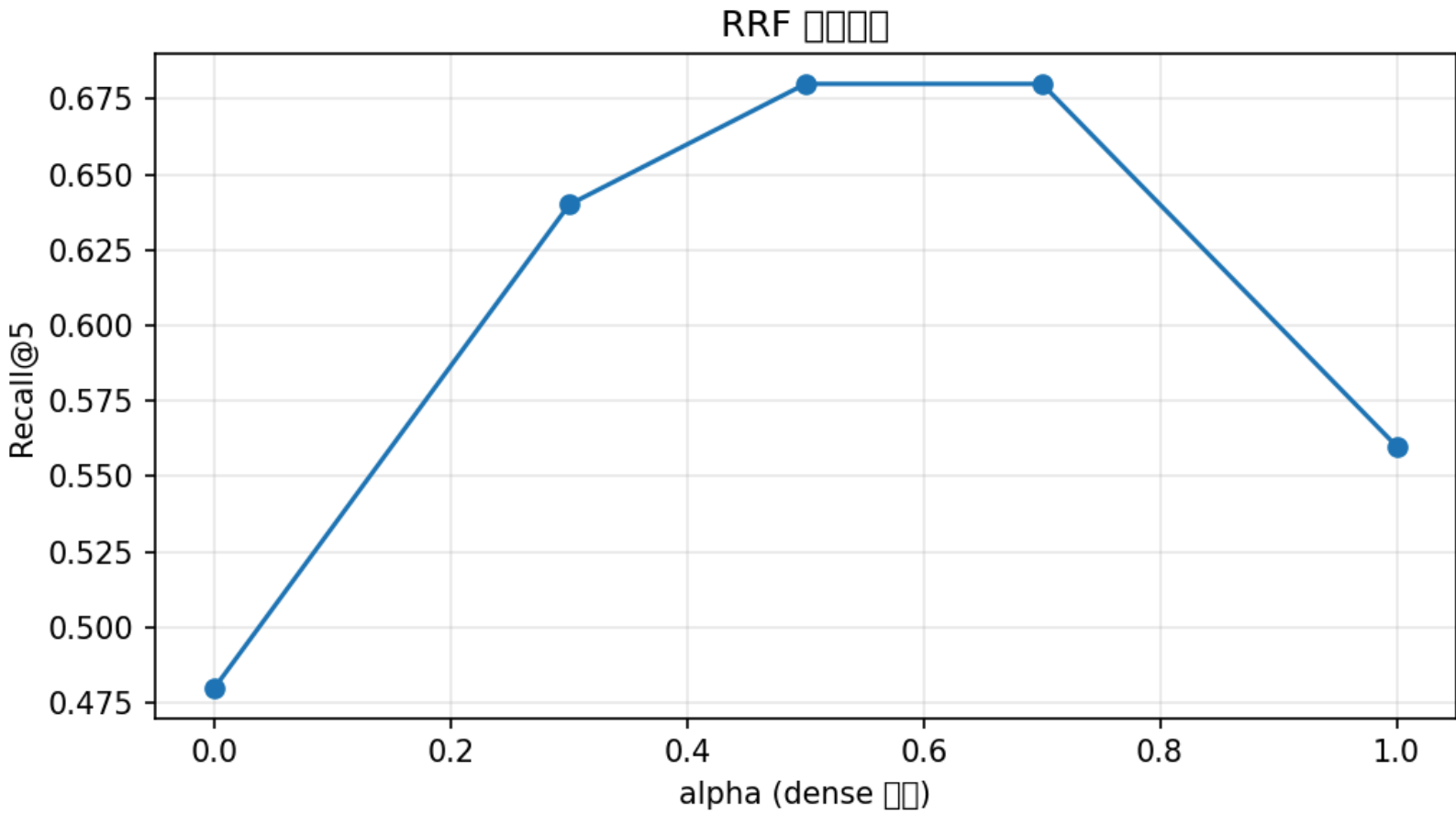


图 3 RRF 权重扫描（ α 为 dense 权重）

子查询分解提升最大（+0.16）；改写与多查询各 +0.08；“多查询 + 双路”反而下降，说明 k 固定时多路融合引入冗余。

RRF 权重扫描显示 $\alpha \in [0.5, 0.7]$ （稠密权重）时 Recall@5 最高（0.68），纯稀疏 0.48、纯稠密 0.56 均低于混合。

说明：本任务未持久化逐条中间查询日志，此处以策略级 Recall 与 RRF 曲线为主要证据（见附录）。

4.4 T4 检索策略对比：稀疏 / 稠密 / 混合

在同一评测集上横向评测 BM25（稀疏）、Embedding+FAISS（稠密）、BM25+Dense 的 RRF 混合检索，计算 Recall@k 与 MRR@10。

指标	dense	sparse	hybrid
Recall@1	0.12	0.16	0.20
Recall@3	0.40	0.36	0.48
Recall@5	0.56	0.48	0.68
Recall@10	0.92	0.72	0.88
MRR@10	0.327	0.309	0.376

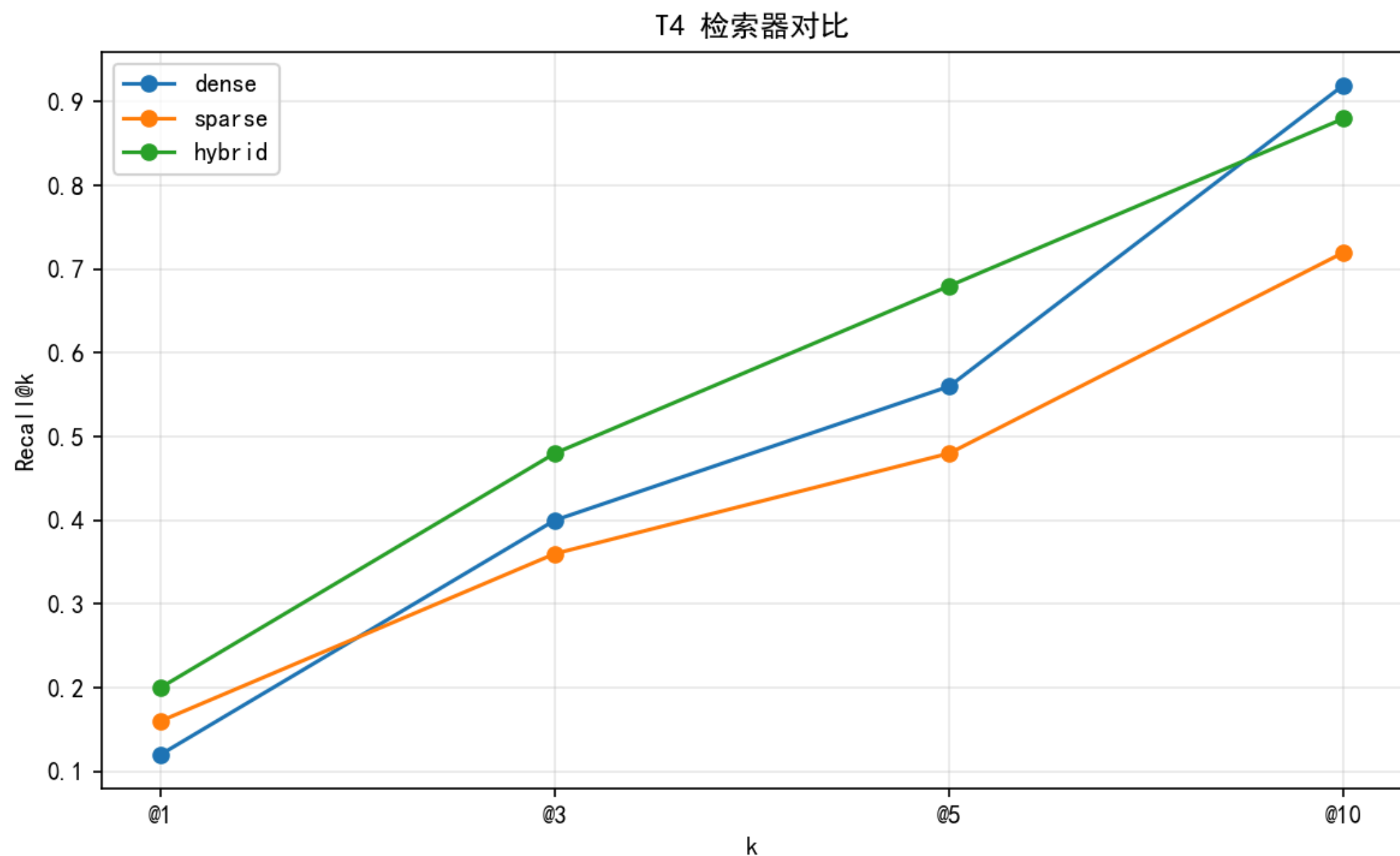


图 4 T4 三种检索器 Recall@k 对比

单边占优对照（自动脚本在 25 条中筛出 5 条单边胜负）：

编号	占优方	问题要点	dense top-3（摘要）	sparse top-3（摘要）
q006	dense	trigger 参数配置	Summarization c003/c002/c001（全中）	混入 human_in_loop / chat_models
q019	dense	v0.x→v1 迁移	migration c002 / whats_new c002 / migration c001	migration c002 / whats_new c002 / migration c003
q020	dense	RAG 评估方案	evaluation c002 / guide c006 / guide c001	evaluation c002 / evaluation c003 / guide c004
q014	sparse	AgentExecutor 导入	migration c004 / classic c004 / migration c002	migration c001 / migration c002 / classic c004
q015	sparse	beforeModel 钩子	whats_new c004 / memory c004 / migration c004	context_eng c003 / middleware c002 / middleware c003

稠密占优：语义相近但字面不重叠（如 q006“trigger 参数”），dense 能按语义命中同文档片段；sparse 被字面词拉偏。

稀疏占优：含专有名词/钩子名（如 q015“beforeModel”“middleware”），BM25 精确命中 middleware_overview；dense 被迁移/记忆文档干扰。

结论：混合检索 Recall@5（0.68）与 MRR@10（0.376）均为最高，Recall@1（0.20）也最高，说明混合在本语料确实带来前位排序增益；但 dense 在 Recall@10（0.92）高于 hybrid（0.88），说明深召回单靠稠密已足够，混合的价值集中在 top 前位。

注：评测集仅 25 条，自动对照仅筛出 5 条单边占优，其余为两路均命中或均未命中，样本量有限。

4.5 T5 检索后优化：重排序与压缩

在 T4 的混合检索之上加入重排与压缩，对比“top-5 召回后取 top-3”各变体的 Recall@3、Context token 数与延迟。

变体	Recall@3	平均 Context token	平均延迟(s)
top3_direct（直接取前 3）	0.48	459.52	0.22
top5_rule_top3（规则重排）	0.48	459.52	0.21
top5_crossencoder_top3	0.40	528.84	20.77
top5_llmjudge_top3	0.48	483.40	1.95
top5_crossencoder_compress	0.40	162.36	17.50

规则重排与 LLM Judge 重排均未改变 Recall@3（仍 0.48）；Cross-Encoder 重排反而降到 0.40，且平均延迟高达 20.77 s（模型逐条加载、CPU 推理）。

压缩把 Context token 从 528.84 降到 162.36（约 -69%），但延迟仍有 17.50 s。

结论：在小规模中文语料上，重排是负优化；压缩能显著省 token，但延迟代价过大，本次场景“省下的 token 不划算”。

4.6 T7 系统评估

对最终系统（header 切分 + 混合检索 top-5 + qwen-plus 生成）做系统评估，覆盖检索与生成两类指标并分层统计。

检索模块（总体 Recall@5 = 0.68、MRR@10 = 0.355、Precision@5 = 0.168）：

等级	Recall@1	Recall@3	Recall@5	Recall@10	MRR@10	Precision@5
L1	0.167	0.500	0.667	0.667	0.339	0.167
L2	0.333	0.667	0.833	0.833	0.514	0.233
L3	0.000	0.333	0.500	0.500	0.153	0.100
L4	0.286	0.429	0.714	0.714	0.405	0.171

生成模块（总体 Correctness = 4.72 / 5、Faithfulness = 4.92 / 5）：

等级	Correctness（1-5）	Faithfulness（1-5）
L1	5.000	5.000
L2	4.833	4.833

L3	5.000	4.833
L4	4.143	5.000

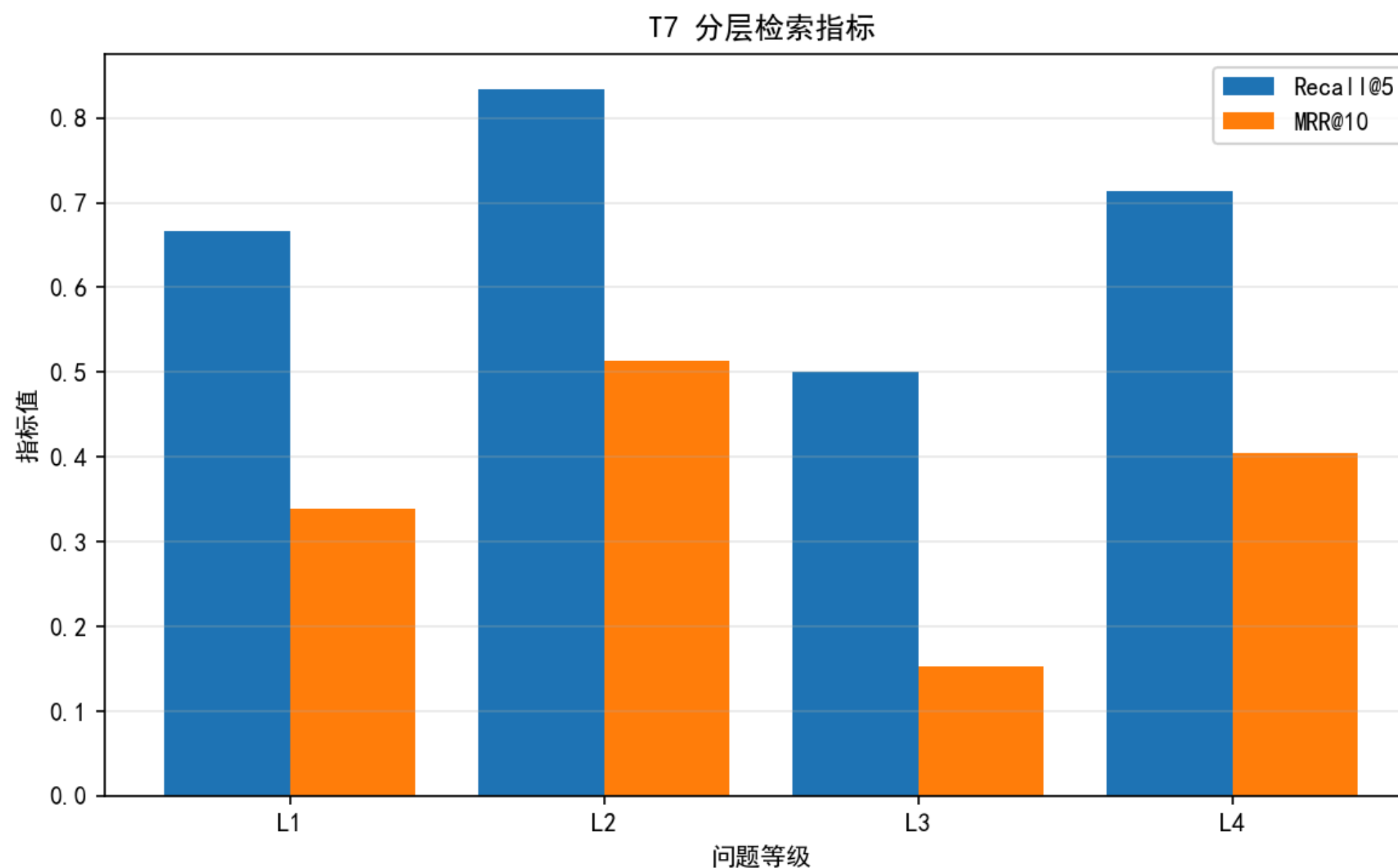


图 5 T7 分层检索指标

失败案例归因：按 Recall@5 排序，最差 5 条为 q013 / q002 / q003 / q023 / q020（Recall@5 均为 0）。逐条核对 gold 片段文本后，发现其主要来自评测集标注噪声而非检索器失效：

- q002 的答案在 05_text_splitters#c006（默认参数），gold 却标为 #c005（结构分割）；
- q003 的答案在 01_framework_architecture#c002（三层抽象），gold 却标为 #c001（标题块）；
- q013 的答案在 01_retrieval_overview#c006（架构对比），gold 却标为 #c005（检索管道）；
- q023 的答案在 03_context_engineering#c004（关键决策），gold 却标为 #c003（中间件作用）；
- q020 的 gold 含不存在的 08_evaluation#c006，且 #c001 为标题块。

此外存在真实生成失败：q019（L4）检索命中 gold 但生成拒答，Correctness 仅 2 分。结论：检索在 L3 最弱（Recall@5=0.50、MRR@10=0.153），生成正确性在 L4 最低（4.14），改进方向为修正 gold_chunk_ids、L3 增加查询分解、L4 使用更强生成策略。

4.7 T8-1 幻觉治理（加分项）

参照课件 9.4.3 的 RAG-HAT 思路，在生成前加一层“信息充分性检测”，先判断召回片段是否足以回答问题，不足则拦截并返回“根据现有资料无法回答”。

变体	平均幻觉率	平均忠实度	拦截率
baseline（无检测层）	0.02	4.92	0.00
with_guard（有检测层）	0.01	4.96	0.08

加入检测层后，平均幻觉率由 0.02 降至 0.01，忠实度由 4.92 升至 4.96，拦截率 8%（2/25 条被判定资料不足）。由于基线幻觉率本身已很低，边际收益有限；检测层的主要价值是对“资料不足”问题做保守拦截，避免无依据作答。

5. 讨论与结论

真正有效的模块：

分块策略：sliding-256-0 把 Recall@5 从默认 header 的 0.56 提升到 0.88，是本次性价比最高的单项优化；

混合检索：Recall@5 0.68、MRR@10 0.376，优于单一稀疏/稠密，尤其提升前位排序；

子查询分解：+0.16 的 Recall@5 提升，RRF 融合在 $\alpha \in [0.5, 0.7]$ 时最优。

负优化 / 无效的模块：

Cross-Encoder 重排：在中文小语料上 Recall@3 不升反降（0.48→0.40），延迟高达 20.77 s；

规则重排与 LLM Judge 重排：Recall@3 零收益；压缩虽省 69% token，但延迟代价过大。

当前最大瓶颈与下一步：

L3 检索是检索端短板（Recall@5=0.50、MRR@10=0.153），应引入查询分解与更强的 rerank（如 bge-reranker-v2-m3 中文版）并缓存模型；

L4 生成正确性最低（4.14），需更强模型或 CoT/要点式提示；评测集 gold_chunk_ids 存在多处错位，是 Recall 被低估的重要来源，应先修正标注再下结论；

评测集仅 25 条，需扩充到 40–60 条并做显著性讨论，避免把波动当结论。

对 RAG 的理解变化：

从“拼一段上下文给 LLM”转变为“检索质量决定上限”。分块与检索融合是低成本、高收益的杠杆，重排与压缩并非万能、需用延迟-质量权衡验证；RAG 主要解决时效性/私有知识的“知不知道”，幻觉治理最终仍要依赖证据约束（先检测再生成）。

6. 附录

6.1 环境与依赖

Python 3.10+；langchain>=1.0.0、langchain-community、langchain-text-splitters、langchain-openai、faiss-cpu>=1.8.0、rank_bm25>=0.2.2、sentence-transformers>=3.0.0、pandas、numpy、matplotlib、tqdm、python-dotenv、pydantic。完整清单见 requirements.txt。随机种子 seed=42。

6.2 复现命令

```
python scripts/run_t1.py --build-index
python scripts/run_t1.py
python scripts/run_t2.py
python scripts/run_t3.py --alpha-scan
```

```
python scripts/run_t4.py --cases
python scripts/run_t5.py
python scripts/run_t7.py
python scripts/run_t8_1.py
```

6.3 指标定义

Recall@k：gold_chunk_ids 中至少一个出现在 top-k 召回结果即算命中， $\text{Recall@k} = \text{命中条数} / \text{总条数}$ ；**MRR@10**：gold chunk 首次出现位置倒数的平均（未命中记 0）；**Precision@5**：top-5 中 gold chunk 占比；**Correctness**：答案与 gold_answer 一致性（LLM-as-Judge，1–5 分）；**Faithfulness**：答案事实是否有召回片段依据（1–5 分）；**幻觉率** = $(5 - \text{Faithfulness}) / 4$ 。

6.4 评测集样例

```
{"id": "q001", "level": "L1", "question": "LangChain v1 中 create_agent 的标准导入路径是什么？",
"gold_answer": "from langchain.agents import create_agent", "gold_chunk_ids":
["02_whats_new_v1#c002", "04_agents_concept#c003"], "evidence_required": true}
```

6.5 实验证据范围说明

- T2 网格以 Recall@5 为主指标，answer-correctness 因 LLM 打分成本未全量计算（脚本支持 --skip-answer）；
- T3 未持久化逐条中间查询日志，报告中以策略级 Recall 与 RRF 曲线为证据；
- T4 自动对照脚本在 25 条中仅筛出 5 条单边占优（3 稠密、2 稀疏），其余为两路均命中或均未命中；
- T7 检索使用 top-5，故 Recall@10 与 Recall@5 数值相同。